**Лабораторная работа №2**

**Численные методы интегрирования**

**Выполнил:** Меркулов Роман Александрович
**Группа:** 6

**Цель работы**

Изучение численных методов интегрирования (методы прямоугольников, трапеций, Симпсона и метод Монте-Карло), их реализация на Python и сравнительный анализ точности и скорости сходимости на примере функции $f(x) = x^2 \sin(x)$ и тройного интеграла $\iiint_0^1 x^2 y^2 z^2 \, dx \, dy \, dz$.

**Ход работы**

| № этапа | Описание этапа | Результат этапа |
|---|---|---|
| 1.1 | Подготовка. Ознакомление с лабораторной работой, ходом решения. Просмотр конспекта. Импорт необходимых библиотек (numpy, matplotlib). | Составлен план работы, начата работа в Jupyter Notebook. |
| 1.2 | Определение подынтегральной функции и вычисление точного значения интеграла аналитически. | Найдена первообразная $F(x) = (2 - x^2)\cos\, x + 2x\sin\, x$, вычислено точное значение $I_{\text{точн}} = \int_0^5 x^2 \sin\, x \, dx \approx -18.113473$. |
| 1.3 | Реализация методов численного интегрирования: левых прямоугольников, правых прямоугольников, средних прямоугольников, трапеций и Симпсона. | Написаны функции для каждого метода, принимающие подынтегральную функцию, пределы интегрирования и число интервалов разбиения. |
| 1.4 | Сравнение методов и построение графиков погрешностей. | Построены графики абсолютной и относительной погрешности в обычном и логарифмическом масштабах. |
| 2.1 | Реализация интегрирования с заданной точностью (апостериорная оценка по правилу Рунге). | Созданы все 5 функций для задания требуемой точности, а также создана универсальная функция *integrate_with_tolerance*, автоматически подбирающая число интервалов для достижения заданной погрешности с коэффициентами 1, 3, 15. |
| 2.2 | Построение графиков зависимости N от требуемой погрешности и их анализ. | Построены графики для группы методов (левые, средние, трапеции) и отдельно для метода Симпсона. |

| | | |
|---|---|---|
| 3.1 | Доимпорт необходимых библиотек и определение функций и других параметров для 3 задания | Импортирована библиотека random для метода Монте-Карло, определены однократные и тройные интегралы |
| 3.2 | Реализация метода Монте-Карло и метода левых прямоугольников для однократного и тройного интегралов. | Написаны функции *left_rectangle_1d, left_rectangle_3d, monte_carlo_1d и monte_carlo_3d.* Для метода Монте-Карло используют генерацию случайных точек и подсчёт попаданий под график функции. |
| 3.3 | Построение графиков и сравнение метода Монте-Карло с методом левых прямоугольников. | Построены графики зависимости относительной погрешности от числа итераций/точек для однократного и тройного интегралов. |

**Выводы**

**Задание 1 (сравнение методов)**

1. Метод Симпсона показал наивысшую точность. При $N = 100$ абсолютная погрешность достигает значений порядка $10^{-8}$
2. Методы средних прямоугольников и трапеций имеют сопоставимую точность. На логарифмическом графике виден одинаковый наклон. Метод средних прямоугольников реализуется проще и в данном примере показывает даже меньшую погрешность, чем трапеции.
3. Методы левых и правых прямоугольников демонстрируют наихудшую точность, их погрешность убывает медленнее всего.

**Задание 2 (интегрирование с заданной точностью)**

1. Метод левых прямоугольников имеет самый пологий наклон на логарифмическом графике. Для достижения точности $10^{-5}$ ему требуется $N \approx 10^7$.
2. Методы средних прямоугольников и трапеций показывают почти одинаковый наклон. Для точности $10^{-5}$ им достаточно $N \approx 10^3$.
3. Метод Симпсона имеет самый крутой наклон. Для достижения точности $10^{-5}$ требуется всего $N \approx 60$, что значительно меньше остальных методов. При точности $10^{-2}$ (2 знака после запятой) достаточно всего $N = 2$ интервалов, что делает метод Симпсона особенно эффективным для умеренных требований к точности.

**Задание 3 (сравнение с методом Монте-Карло)**

1. Для однократного интеграла метод левых прямоугольников сходится быстрее метода Монте-Карло. Наклон левых прямоугольников круче, что соответствует более высокому порядку точности. Метод Монте-Карло плохо применим к однократным интегралам, так как он основан на вероятностной природе и допускает большую погрешность. В одномерном случае детерминированный метод эффективнее.
2. Для тройного интеграла ситуация меняется. Левые прямоугольники требуют $N^3$ вычислений для достижения точности, в то время как Монте-Карло — только $N$. При одинаковом количестве вычислений (например, 1000) погрешность обоих методов сопоставима, хотя по общей динамике метод Монте-Карло выигрывает. В

трёхмерном случае метод Монте-Карло становится предпочтительным из-за линейного роста вычислительных затрат.

**Импортируемые функции сторонних библиотек**

| Библиотека | Функция | Назначение | Основные параметры |
|---|---|---|---|
| matplotlib | plt.figure() | Создание нового рисунка | figsize - размер рисунка |
| matplotlib | plt.plot() | Построение графика | массив значений x, y, color - цвет, linewidth - толщина, label - подпись |
| matplotlib | plt.semilogy() | График с логарифмической шкалой по оси Y | массив значений x, y , color, linewidth, label |
| matplotlib | plt.loglog() | График с логарифмической шкалой по обеим осям | x, y, color, linewidth, label |
| matplotlib | plt.title() | Заголовок графика | сам текст, fontsize, fontweight |
| matplotlib | plt.xlabel() | Подпись оси X | сам текст, fontsize |
| matplotlib | plt.ylabel() | Подпись оси Y | сам текст, fontsize |
| matplotlib | plt.grid() | Сетка на графике | Visible – видимость, alpha - прозрачность |
| matplotlib | plt.legend() | Легенда | fontsize |
| matplotlib | plt.tight_layout() | Автоматическая подгонка элементов | — |
| matplotlib | plt.show() | Отображение графика | — |
| random | random.uniform() | Генерация случайного числа на интервале | a, b |
| numpy | np.abs() | Модуль числа | число |
| numpy | np.sin() | Синус | число |
| numpy | np.cos() | Косинус | число |
| numpy | np.arange() | Создание массива с заданным шагом | start, stop, step |
| numpy | np.mean() | Среднее арифметическое | массив чисел |